```
import sqlite3
from contextlib import contextmanager
from config import DB_PATH


@contextmanager
def get_conn():
    conn = sqlite3.connect(DB_PATH)
    conn.row_factory = sqlite3.Row
    conn.execute("PRAGMA foreign_keys =
ON")
    try:
        yield conn
        conn.commit()
    finally:
        conn.close()


def init_db():
    with get_conn() as conn:
        conn.executescript("""
            CREATE TABLE IF NOT EXISTS
users (
                user_id INTEGER PRIMARY
KEY,
                full_name TEXT,
                username TEXT,
                balance INTEGER DEFAULT 0,
                tariff_id INTEGER,
                tariff_expires TEXT,
```

```
                referred_by INTEGER,
                created_at TEXT DEFAULT
(datetime('now'))
            );
            CREATE TABLE IF NOT EXISTS
tariffs (
                tariff_id INTEGER PRIMARY
KEY AUTOINCREMENT,
                name TEXT NOT NULL,
                price INTEGER NOT NULL,
                duration_days INTEGER NOT
NULL,
                category TEXT DEFAULT
'ertak',
                description TEXT,
                is_active INTEGER DEFAULT 1
            );
            CREATE TABLE IF NOT EXISTS
books (
                book_id INTEGER PRIMARY KEY
AUTOINCREMENT,
                title TEXT NOT NULL,
                author TEXT,
                file_id TEXT,
                reward INTEGER DEFAULT 0,
                category TEXT DEFAULT
'ertak',
                is_active INTEGER DEFAULT
1,
                created_at TEXT DEFAULT
(datetime('now'))
            );
            CREATE TABLE IF NOT EXISTS
reading_progress (
```

```
                id INTEGER PRIMARY KEY
AUTOINCREMENT,
                user_id INTEGER NOT NULL,
                book_id INTEGER NOT NULL,
                status TEXT DEFAULT
'started',
                reward_paid INTEGER DEFAULT
0,
                started_at TEXT DEFAULT
(datetime('now')),
                finished_at TEXT,
                UNIQUE(user_id, book_id)
            );
            CREATE TABLE IF NOT EXISTS
deposits (
                deposit_id INTEGER PRIMARY
KEY AUTOINCREMENT,
                user_id INTEGER NOT NULL,
                tariff_id INTEGER NOT NULL,
                amount INTEGER NOT NULL,
                receipt_file_id TEXT,
                status TEXT DEFAULT
'pending',
                created_at TEXT DEFAULT
(datetime('now')),
                decided_at TEXT
            );
            CREATE TABLE IF NOT EXISTS
withdrawals (
                withdrawal_id INTEGER
PRIMARY KEY AUTOINCREMENT,
                user_id INTEGER NOT NULL,
                amount INTEGER NOT NULL,
                card_number TEXT,
```

```
                status TEXT DEFAULT
'pending',
                created_at TEXT DEFAULT
(datetime('now')),
                decided_at TEXT
            );
            CREATE TABLE IF NOT EXISTS
daily_tasks (
                task_id INTEGER PRIMARY KEY
AUTOINCREMENT,
                task_date TEXT NOT NULL
UNIQUE,
                title TEXT NOT NULL,
                description TEXT,
                is_active INTEGER DEFAULT
1,
                created_at TEXT DEFAULT
(datetime('now'))
            );
            CREATE TABLE IF NOT EXISTS
task_completions (
                id INTEGER PRIMARY KEY
AUTOINCREMENT,
                user_id INTEGER NOT NULL,
                task_id INTEGER NOT NULL,
                reward INTEGER NOT NULL,
                completed_at TEXT DEFAULT
(datetime('now')),
                UNIQUE(user_id, task_id)
            );
            CREATE TABLE IF NOT EXISTS
dice_plays (
                id INTEGER PRIMARY KEY
AUTOINCREMENT,
```

```
                user_id INTEGER NOT NULL,
                week_key TEXT NOT NULL,
                amount_won INTEGER DEFAULT
0,
                played_at TEXT DEFAULT
(datetime('now')),
                UNIQUE(user_id, week_key)
            );
        """)
        # Migrations
        for sql in [
            "ALTER TABLE tariffs ADD COLUMN
category TEXT DEFAULT 'ertak'",
            "ALTER TABLE books ADD COLUMN
category TEXT DEFAULT 'ertak'",
            "ALTER TABLE users ADD COLUMN
referred_by INTEGER",
        ]:
            try:
                conn.execute(sql)
            except Exception:
                pass
        # Mavjud tariflar uchun kategoriya
belgilash
        conn.execute("UPDATE tariffs SET
category='ertak' WHERE tariff_id=1 AND
(category IS NULL OR category='')")
        conn.execute("UPDATE tariffs SET
category='hikoya' WHERE tariff_id=2 AND
(category IS NULL OR category='')")
        conn.execute("UPDATE tariffs SET
category='roman' WHERE tariff_id=3 AND
(category IS NULL OR category='')")
        conn.execute("UPDATE tariffs SET
```

```
category='chet_el' WHERE tariff_id=4 AND
(category IS NULL OR category='')")


# ---- USERS ----
def upsert_user(user_id, full_name,
username, referred_by=None):
    with get_conn() as conn:
        conn.execute(
            """INSERT INTO users (user_id,
full_name, username, referred_by)
               VALUES (?, ?, ?, ?)
               ON CONFLICT(user_id) DO
UPDATE SET full_name=excluded.full_name,
username=excluded.username""",
            (user_id, full_name, username,
referred_by),
        )

def get_user(user_id):
    with get_conn() as conn:
        return conn.execute("SELECT * FROM
users WHERE user_id=?",
(user_id,)).fetchone()

def change_balance(user_id, delta):
    with get_conn() as conn:
        conn.execute("UPDATE users SET
balance = balance + ? WHERE user_id=?",
(delta, user_id))

def set_user_tariff(user_id, tariff_id,
expires):
    with get_conn() as conn:
```

```
        conn.execute(
            "UPDATE users SET tariff_id=?,
tariff_expires=? WHERE user_id=?",
            (tariff_id, expires, user_id),
        )

def get_referrer(user_id):
    with get_conn() as conn:
        row = conn.execute("SELECT
referred_by FROM users WHERE user_id=?",
(user_id,)).fetchone()
        return row["referred_by"] if row
else None

# ---- TARIFFS ----
def add_tariff(name, price, duration_days,
category='ertak', description=''):
    with get_conn() as conn:
        conn.execute(
            "INSERT INTO tariffs (name,
price, duration_days, category,
description) VALUES (?,?,?,?,?)",
            (name, price, duration_days,
category, description),
        )

def list_tariffs(active_only=True):
    with get_conn() as conn:
        q = "SELECT * FROM tariffs" + ("
WHERE is_active=1" if active_only else "")
+ " ORDER BY price"
        return conn.execute(q).fetchall()

def get_tariff(tariff_id):
```

```
    with get_conn() as conn:
        return conn.execute("SELECT * FROM
tariffs WHERE tariff_id=?",
(tariff_id,)).fetchone()

def delete_tariff(tariff_id):
    with get_conn() as conn:
        conn.execute("UPDATE tariffs SET
is_active=0 WHERE tariff_id=?",
(tariff_id,))

# ---- BOOKS ----
def add_book(title, author, file_id,
reward, category='ertak'):
    with get_conn() as conn:
        conn.execute(
            "INSERT INTO books (title,
author, file_id, reward, category) VALUES
(?,?,?,?,?)",
            (title, author, file_id,
reward, category),
        )

def list_books(category=None,
active_only=True):
    with get_conn() as conn:
        if category:
            q = "SELECT * FROM books WHERE
category=?"
            if active_only:
                q += " AND is_active=1"
            return conn.execute(q + " ORDER
BY book_id DESC", (category,)).fetchall()
        q = "SELECT * FROM books"
```

```
        if active_only:
            q += " WHERE is_active=1"
        return conn.execute(q + " ORDER BY
book_id DESC").fetchall()

def get_book(book_id):
    with get_conn() as conn:
        return conn.execute("SELECT * FROM
books WHERE book_id=?",
(book_id,)).fetchone()

def delete_book(book_id):
    with get_conn() as conn:
        conn.execute("UPDATE books SET
is_active=0 WHERE book_id=?", (book_id,))

# ---- READING ----
def start_reading(user_id, book_id):
    with get_conn() as conn:
        conn.execute(
            "INSERT INTO reading_progress
(user_id, book_id) VALUES (?,?) ON
CONFLICT(user_id, book_id) DO NOTHING",
            (user_id, book_id),
        )

def get_progress(user_id, book_id):
    with get_conn() as conn:
        return conn.execute(
            "SELECT * FROM reading_progress
WHERE user_id=? AND book_id=?", (user_id,
book_id)
        ).fetchone()
```

```
def finish_reading(user_id, book_id):
    with get_conn() as conn:
        conn.execute(
            "UPDATE reading_progress SET
status='finished',
finished_at=datetime('now') WHERE user_id=?
AND book_id=?",
            (user_id, book_id),
        )

def mark_reward_paid(user_id, book_id):
    with get_conn() as conn:
        conn.execute(
            "UPDATE reading_progress SET
reward_paid=1 WHERE user_id=? AND
book_id=?", (user_id, book_id)
        )

# ---- DEPOSITS ----
def create_deposit(user_id, tariff_id,
amount, receipt_file_id):
    with get_conn() as conn:
        cur = conn.execute(
            "INSERT INTO deposits (user_id,
tariff_id, amount, receipt_file_id) VALUES
(?,?,?,?)",
            (user_id, tariff_id, amount,
receipt_file_id),
        )
        return cur.lastrowid

def get_deposit(deposit_id):
    with get_conn() as conn:
        return conn.execute("SELECT * FROM
```

```
deposits WHERE deposit_id=?",
(deposit_id,)).fetchone()

def list_pending_deposits():
    with get_conn() as conn:
        return conn.execute("SELECT * FROM
deposits WHERE status='pending' ORDER BY
created_at").fetchall()

def decide_deposit(deposit_id, status):
    with get_conn() as conn:
        conn.execute(
            "UPDATE deposits SET status=?,
decided_at=datetime('now') WHERE
deposit_id=?", (status, deposit_id)
        )

# ---- WITHDRAWALS ----
def create_withdrawal(user_id, amount,
card_number):
    with get_conn() as conn:
        cur = conn.execute(
            "INSERT INTO withdrawals
(user_id, amount, card_number) VALUES
(?,?,?)",
            (user_id, amount, card_number),
        )
        return cur.lastrowid

def get_withdrawal(withdrawal_id):
    with get_conn() as conn:
        return conn.execute("SELECT * FROM
withdrawals WHERE withdrawal_id=?",
(withdrawal_id,)).fetchone()
```

```
def list_pending_withdrawals():
    with get_conn() as conn:
        return conn.execute("SELECT * FROM
withdrawals WHERE status='pending' ORDER BY
created_at").fetchall()

def decide_withdrawal(withdrawal_id,
status):
    with get_conn() as conn:
        conn.execute(
            "UPDATE withdrawals SET
status=?, decided_at=datetime('now') WHERE
withdrawal_id=?", (status, withdrawal_id)
        )

# ---- DAILY TASKS ----
def add_daily_task(task_date, title,
description=''):
    with get_conn() as conn:
        conn.execute(
            "INSERT OR REPLACE INTO
daily_tasks (task_date, title, description)
VALUES (?,?,?)",
            (task_date, title,
description),
        )

def get_today_task(task_date):
    with get_conn() as conn:
        return conn.execute(
            "SELECT * FROM daily_tasks
WHERE task_date=? AND is_active=1",
(task_date,)
```

```
        ).fetchone()

def get_task(task_id):
    with get_conn() as conn:
        return conn.execute("SELECT * FROM
daily_tasks WHERE task_id=?",
(task_id,)).fetchone()

def complete_task(user_id, task_id,
reward):
    with get_conn() as conn:
        try:
            conn.execute(
                "INSERT INTO
task_completions (user_id, task_id, reward)
VALUES (?,?,?)",
                (user_id, task_id, reward),
            )
            return True
        except Exception:
            return False

def has_completed_task(user_id, task_id):
    with get_conn() as conn:
        return conn.execute(
            "SELECT 1 FROM task_completions
WHERE user_id=? AND task_id=?", (user_id,
task_id)
        ).fetchone() is not None

# ---- DICE ----
def get_dice_play(user_id, week_key):
    with get_conn() as conn:
        return conn.execute(
```

```
            "SELECT * FROM dice_plays WHERE
user_id=? AND week_key=?", (user_id,
week_key)
        ).fetchone()

def save_dice_play(user_id, week_key,
amount_won):
    with get_conn() as conn:
        conn.execute(
            "INSERT INTO dice_plays
(user_id, week_key, amount_won) VALUES
(?,?,?)",
            (user_id, week_key,
amount_won),
        )
```